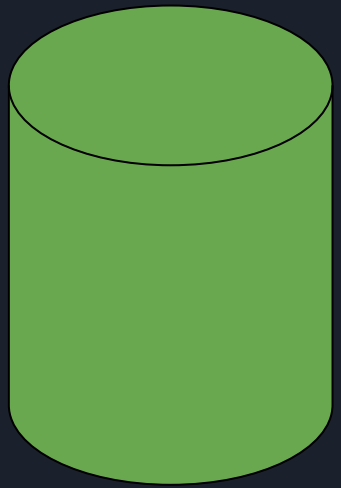
GESTIÓN DE DATOS

PL/SQL - TRIGGERS

RECORDERIS...



¿QUÉ APRENDEREMOS HOY?



- Qué son los triggers (o disparadores) y para qué se usan
- Tipos de triggers
- Cómo éstos pueden ayudar a automatizar tareas operativas

Pregunta: ¿Qué es un trigger?



DEFINICIÓN: TRIGGER

Código pl/sql que se ejecuta de forma automática para reaccionar ante cambios en los datos, no para iniciar procesos.

NOTA: No aplica para operaciones de consulta (select)

TRIGGERS: Ejemplos

Ejemplo 1:

En una papelería se realiza una venta, la cual implica un *insert* en la tabla de ventas. Este insert ejecutará automáticamente un código pl/sql (también puede ser una función o un procedimiento almacenado) que hará un *update* en la tabla de inventario

TRIGGERS: Ejemplos

Ejemplo 2:

En un sistema de logística se actualiza el estado de los pedidos, por ejemplo "abierto", "en proceso", "en despacho", "finalizado". Cuando el estado cambie a "finalizado" mediante un *update*, realizar un *insert* en una tabla de encuestas de satisfacción de modo que posteriormente se pueda enviar al cliente para su calificación

TRIGGERS: Ejemplos

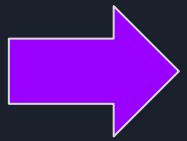
Ejemplo 3:

En un sistema de gestión académica un estudiante abandona un curso, lo cual implica un *delete* sobre la tabla que relaciona al estudiante con el curso. Esta operación puede generar un *insert* en una tabla de históricos, de modo que se pueda tener referencia de qué cursos han sido abandonados por cuáles estudiantes

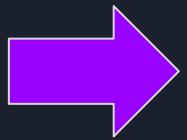
¿Para qué crear triggers?

- Automatizar algunas operaciones (como se mencionó en los ejemplos anteriores)
- Auditoría de quién modifica qué datos, cuándo y desde donde
- Tener historia de los estados de un proceso (similar a un diagrama de estados)

Tiempo de los triggers

BEFORE 

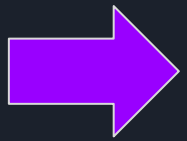
El código pl/sql se ejecuta primero, y luego se hace la operación de update, insert o delete que disparó el trigger

AFTER 

Primero se hace la operación de update, insert o delete y luego se ejecuta el código pl/sql del trigger

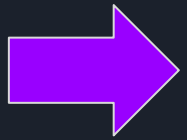
¿Cuando usar cual?

BEFORE (validar y corregir)



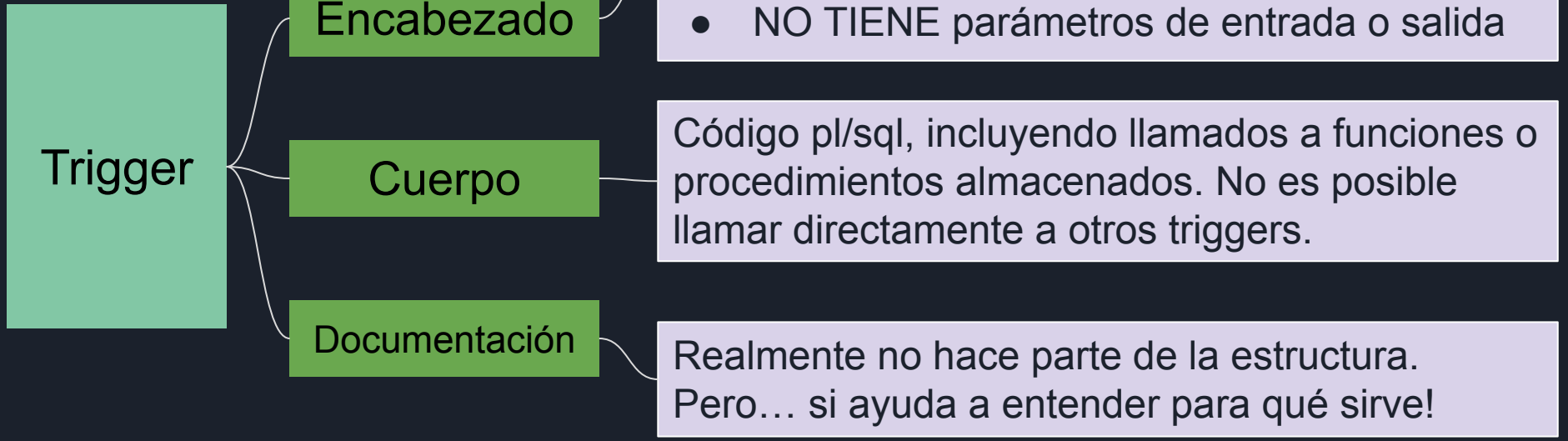
Validar datos o ajustar valores por defecto antes de guardar

AFTER (registrar)



Es el más común. Para auditoría, históricos o notificaciones

Estructura de un trigger



Estructuras OLD y NEW

Permiten conocer el estado del registro antes y/o después del cambio.

OLD y NEW: Ejemplos (update)

1

ID	FULL_NAME	BIRTH_DATE	PROFESSION
1	SELENA GOMEZ		

2

```
update singers set profession = 'ARTIST' where id = 1;
```

3

```
:old.id ⇒ 123                          :new.id ⇒ 123
:old.full_name ⇒ 'SELENA GOMEZ'        :new.full_name = 'SELENA GOMEZ'
:old.birth_date ⇒ null                 :new.birth_date ⇒ null
:old.profession ⇒ null                 :new.profession ⇒ 'ARTIST'
```

4

ID	FULL_NAME	BIRTH_DATE	PROFESSION
1	SELENA GOMEZ		ARTIST

OLD y NEW: Ejemplos (insert)

1

ID	FULL_NAME	BIRTH_DATE	PROFESSION

2

```
insert into singers (full_name, birth_date)
values ('SELENA GOMEZ', '1992-07-22')';
```

3

```
:old.id ⇒ null                    :new.id ⇒ 1  (asumiendo auto increment)
:old.full_name ⇒ null             :new.full_name = 'SELENA GOMEZ'
:old.birth_date ⇒ null            :new.birth_date ⇒ '1992-07-22'
:old.profession ⇒ null            :new.profession ⇒ null
```

4

ID	FULL_NAME	BIRTH_DATE	PROFESSION
1	SELENA GOMEZ	1992-07-22	

OLD y NEW: Ejemplos (delete)

1

ID	FULL_NAME	BIRTH_DATE	PROFESSION
1	SELENA GOMEZ	1992-07-22	

2

```
delete from singers where id = 1;
```

3

```
:old.id ⇒ 1                          :new.id ⇒ null
:old.full_name ⇒ 'SELENA GOMEZ'      :new.full_name = null
:old.birth_date ⇒ '1992-07-22'       :new.birth_date ⇒ null
:old.profession ⇒ null               :new.profession ⇒ null
```

4

ID	FULL_NAME	BIRTH_DATE	PROFESSION

Ejemplo: Auditar cambios en "world"

Para monitorear los ***cambios de población*** en las ciudades, se creará una tabla para guardar la historia de cada cambio.

```
create table cities_history(
    id int(11) not null auto_increment,
    city_id int(11) not null,
    event_date datetime not null,
    event_user varchar(100) not null,
    event_action varchar(30) not null,
    value_before varchar(255) not null,
    value_after varchar(255) not null,
    primary key(id)
);
```

Ejemplo: Auditar cambios en "world"

```
delimiter $$

create trigger tr_audit_city_population
after update on cities
for each row
begin

    insert into cities_history(city_id, event_date, event_user,
event_action, value_before, value_after)
    values (old.id, now(), user(),
'UPDATE POPULATION', old.Population, new.Population );

end;
$$
delimiter ;
```

Nombre del trigger

Tiempo y tipo de operación: este código se ejecuta DESPUÉS de actualizar la tabla

La tabla a monitorear es "cities"

"old" y "new" son estructuras de datos que permiten obtener los valores antes y después de actualizar

Ejemplo: Auditar cambios en "world"

Antes de actualizar

```
MariaDB [world]> select * from city where id = 2259;
+------+-----------+-------------+-----------+------------+
| ID   | Name      | CountryCode | District  | Population |
+------+-----------+-------------+-----------+------------+
| 2259 | Medellín  | COL         | Antioquia |    1861265 |
+------+-----------+-------------+-----------+------------+
1 row in set (0,001 sec)

MariaDB [world]> select * from city_history;
Empty set (0,000 sec)
```

```
MariaDB [world]> update city set Population = 2590000 where id = 2259;
Query OK, 1 row affected (0,006 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

Después de actualizar

```
MariaDB [world]> select * from city where id = 2259;
+------+-----------+-------------+-----------+------------+
| ID   | Name      | CountryCode | District  | Population |
+------+-----------+-------------+-----------+------------+
| 2259 | Medellín  | COL         | Antioquia |    2590000 |
+------+-----------+-------------+-----------+------------+
1 row in set (0,000 sec)

MariaDB [world]> select * from city_history;
+----+---------+---------------------+----------------+------------------+--------------+-------------+
| id | city_id | event_date          | event_user     | event_action     | value_before | value_after |
+----+---------+---------------------+----------------+------------------+--------------+-------------+
|  1 |    2259 | 2024-03-30 19:28:25 | root@172.19.0.1 | UPDATE POPULATION |      1861265 |     2590000 |
+----+---------+---------------------+----------------+------------------+--------------+-------------+
1 row in set (0,000 sec)
```

EJERCICIO (1/4)

NOTA

En esta presentación se proporcionó un ejemplo para triggers tipo "after update", que son eventos que se ejecutan después de actualizar. Sin embargo, y a propósito, no se proporcionaron ejemplos para después de borrar o después de insertar, ni tampoco ejemplos tipo "before", ya que quedarán como ejercicio de consulta.

EJERCICIO (2/4)

Ejercicio 1:

Crear un trigger que monitoree registros borrados sobre la tabla "cities". En la tabla de auditoría debe registrarse así:

- En el campo "event_action" debe quedar el texto "DELETE CITY"
- En el campo "value_before" debe quedar el nombre de la ciudad, código del país, nombre del distrito y población que estaban antes de borrar el registro, como un texto separado por coma
- El campo "value_after" debe quedar con valor "null"
- En el campo "city_id" debe quedar el id de la ciudad que se borra

EJERCICIO (3 / 4)

Ejercicio 2:

Crear un trigger que monitoree registros insertados sobre la tabla "cities". En la tabla de auditoría debe registrarse así:

- En el campo "event_action" debe quedar el texto "CREATE CITY"
- En el campo "value_after" debe quedar el nombre de la ciudad, código del país, nombre del distrito y población que se están insertando, como un texto separado por coma
- El campo "value_before" debe quedar con valor "null"
- En el campo "city_id" debe quedar el id de la ciudad que se inserta

EJERCICIO (4/4)

Ejercicio 3:

Crear un trigger que, al momento de actualizar la población, sume 1 al valor que se especifica en el update. La tabla de auditoría debe quedar igual que en el ejemplo dado en esta presentación. Es decir, si la población que se especifica en el update es de 100000, entonces el valor que debe quedar en la tabla es 100001

- NOTA: Este trigger es tipo "before update".

¿PREGUNTAS?

